

Lab 10: Micro labs de AWS con Terraform

Diagramas de arquitectura, implementación con Terraform, RBAC y seguridad DevOps sobre GitLab CI, alineados con el AWS Well-Architected Framework.

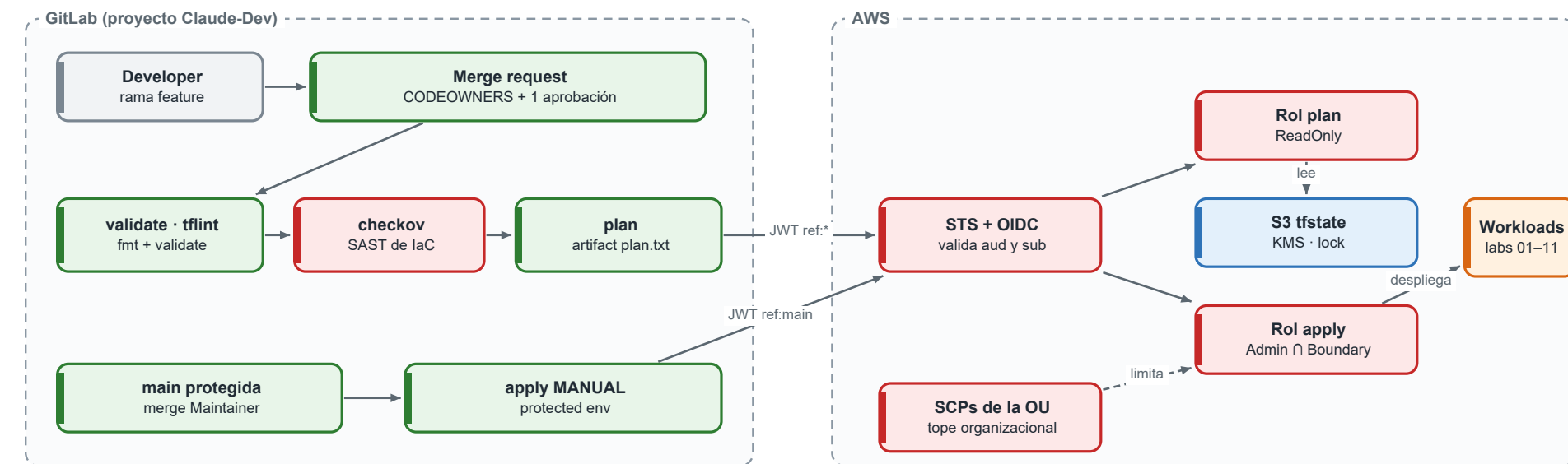
#	Micro lab	Servicios principales	Arquitectura
00	Bootstrap	S3 state, KMS, IAM OIDC, Budgets	Plataforma
01	API serverless	API Gateway REST, Lambda, DynamoDB, Cognito, WAF	Serverless
02	AppSync GraphQL	AppSync, resolvers JS, DynamoDB	Serverless GraphQL
03	EventBridge event-driven	EventBridge, Scheduler, SQS, Lambda	Event-driven
04	Chatbot Bedrock	HTTP API, Lambda, Bedrock, Guardrails	IA generativa
05	Pipeline IA de documentos	S3, Step Functions, Textract, Comprehend, Bedrock	IA orquestada
06	Web de 3 niveles	VPC, ALB, WAF, ASG, RDS Multi-AZ	3 niveles
07	SRE observabilidad	CloudWatch SLO, Synthetics, Q chat apps, FIS	SRE
08	Neptune grafos	Neptune Serverless, Lambda VPC	Grafos
09	Migración MGN	AWS MGN, KMS, IAM	Migración
10	CloudFormation StackSets	CloudFormation StackSets, Organizations	Multi-cuenta
11	Seguridad y gobernanza	CloudTrail, Config, GuardDuty, Security Hub, SCP	Gobernanza

Código fuente: 03-cloud-e-infraestructura/lab10-terraform-aws/ (módulos en modules/, labs en microlabs/NN-nombre/). Cada lab incluye README con instrucciones, terraform.tfvars.example y ci.yml. Todos pasan terraform validate (Terraform 1.13 · AWS provider 6.x); no se han desplegado en una cuenta real.

Parte 1 · Cómo hacer el RBAC y la seguridad DevOps

El acceso se diseña en dos planos que se refuerzan: quién puede cambiar el código (roles de GitLab, ramas y entornos protegidos) y con qué identidad se despliega (roles IAM asumidos por OIDC, nunca llaves). Ninguna persona necesita permisos de escritura permanentes en AWS: el pipeline aplica lo que se aprobó en el merge request.

1.1 Flujo GitOps con OIDC y separación de funciones



■ Externo / usuario ■ Red ■ Cómputo ■ Datos / storage ■ Integración / API ■ IA / ML ■ Seguridad / identidad ■ Operación / observabilidad

La condición sub del token es la llave del modelo: `project_path:grupo/proyecto:ref_type:branch:ref:main` solo existe en pipelines de la rama protegida, así que una rama feature nunca puede asumir el rol de apply.

1.2 Matriz RBAC (GitLab ↔ AWS)

Rol	GitLab (proyecto)	AWS (IAM role / permission set)	Puede hacer	No puede hacer
Developer	Developer: push a ramas, abrir MR, ver plan	lab10-dev-sec-developer (boundary + ABAC), 8 h	Workloads serverless/EC2 pequeñas, recursos de su Team, roles lab10-* con boundary	Merge a main, apply en CI, IAM users/keys, apagar seguridad, instancias grandes
Platform / SRE	Maintainer: merge, ejecutar apply/destroy	lab10-dev-sec-platform-engineer, 4 h	Infraestructura compartida (VPC, módulos), operar pipelines y runbooks	Cambiar SCPs o desactivar CloudTrail/GuardDuty (SCP)
Security	Owner del grupo security; CODEOWNER de lab 10/11	Cuenta de seguridad + lab10-security-audit (StackSets)	SCPs, baseline, Security Hub, respuesta a incidentes	Desplegar workloads
Auditor	Reporter (solo lectura)	lab10-dev-sec-auditor, 4 h	Leer configuración, Config, CloudTrail y Security Hub	Cualquier escritura
FinOps	Reporter	lab10-dev-sec-finops, 4 h	Billing, Budgets, Cost Explorer	Cambios de infraestructura
CI plan	Job plan (cualquier rama/MR)	lab10-dev-gitlab-plan (OIDC sub ref:*)	Leer AWS y el estado; escribir el lock	Crear o modificar recursos
CI apply	Job apply (solo main, manual, protected env)	lab10-dev-gitlab-apply (OIDC sub ref:main) ∩ boundary	Aplicar planes aprobados	Salir de las regiones permitidas; tocar el estado; crear users/keys
Break-glass	N/A (fuera del pipeline)	lab10-dev-sec-break-glass-admin, 1 h, MFA de menos de 1 h	Recuperación en emergencias	Usarse sin alerta: cada AssumeRole notifica a seguridad

Recomendación para producción: centralizar personas en IAM Identity Center (SSO con el IdP de la universidad o la empresa) y mapear grupos a permission sets equivalentes a los roles del lab 11. Usa una cuenta por entorno (dev/test/prod) dentro de AWS Organizations y propaga el baseline con StackSets (lab 10).

1.3 Capas de guardrails (defensa en profundidad)

El permiso efectivo es la intersección de todas las capas: una acción debe estar permitida en cada una y no estar denegada en ninguna.

Capa (de afuera hacia adentro)	Dónde se define	Qué limita	Micro lab
1. SCP (Organizations)	policies/scp/*.json	Máximo permitido para TODA la cuenta, incluido el admin: regiones, cifrado, root, servicios de seguridad	11 / 10
2. Permissions boundary	aws_iam_policy.boundary / developer_boundary	Máximo para un rol concreto aunque tenga AdministratorAccess o PowerUser	00 / 11
3. Política de identidad (RBAC)	aws_iam_role.rbac + managed policies	Lo que la función necesita: developer, platform, auditor, finops, break-glass	11
4. ABAC (etiquetas)	abac-ec2-team.json	Restringe dentro del rol a los recursos del propio equipo (Team)	11
5. Política de recurso	Políticas de S3/SQS/SNS/KMS/bus	Quién puede usar el recurso: aws:SourceArn, aws:PrincipalOrgID, TLS-only	01–11
6. Red	Security groups encadenados, subnets aisladas, endpoints	Alcance de red mínimo; sin SSH; datos sin ruta a Internet	06 / 08 / 09

1.4 Controles DevSecOps del pipeline

Control	Implementación en lab10	Dónde
Sin secretos de larga duración	OIDC id_tokens → AssumeRoleWithWebIdentity; condiciones aud + sub	.gitlab-ci.yml (.aws_oidc), lab 00
Separación de funciones	Rol plan (lectura) ≠ rol apply (rama protegida); apply manual	lab 00, ci.yml de cada lab
Revisión obligatoria	Protected branch main, MR approvals, CODEOWNERS en modules/ y labs 10/11	Settings → Repository / Merge requests
Escaneo de IaC	terraform fmt/validate, tfint, checkov (reporte JUnit en el MR)	Etapas validate y security
Plan inmutable	apply usa el plan.tfplan exacto del job plan (artifact)	.tf_plan → .tf_apply
Concurrencia segura	resource_group por lab/entorno + use_lockfile de S3	.tf_apply / .tf_destroy
Estado protegido	S3 SSE-KMS, versionado, TLS-only, boundary que impide borrar el bucket	lab 00
Variables sensibles	tfvars como variables File protegidas; masked para IDs/ARNs; *.tfvars en .gitignore	Settings → CI/CD → Variables
Entornos protegidos	environment dev/<lab>, prod/<lab>; solo Maintainers despliegan	Settings → CI/CD → Protected environments
Trazabilidad	Etiquetas por default_tags (Project, Owner, MicroLab, CostCenter); CloudTrail	versions.tf, lab 11
Detección continua	GuardDuty, Security Hub, Config → EventBridge → SNS/Slack	labs 07, 10, 11
Costo como control	AWS Budgets, concurrencia reservada, tokens máximos, destroy manual	labs 00, 04, 05

1.5 Variables a configurar en GitLab (Settings → CI/CD → Variables)

Variable	Tipo	Protegida	Enmascarada	Valor / origen
TF_STATE_BUCKET	Variable	No	No	output tf_state_bucket (lab 00)
AWS_PLAN_ROLE_ARN	Variable	No	Sí	output aws_plan_role_arn (lab 00)
AWS_APPLY_ROLE_ARN	Variable	Sí	Sí	output aws_apply_role_arn (lab 00)
AWS_REGION / TF_ENV	Variable	No	No	us-east-1 / dev (environment scope para prod)
LAB01_TFVARS ... LAB11_TFVARS	File	Sí	No	Contenido de terraform.tfvars de cada lab
DESTROY_LAB	Variable (al ejecutar)	No	No	Nombre del lab a destruir, p. ej. 06-web3tier-alb-asg-rds
TF_VAR_*	Variable	Según dato	Según dato	Sobrescribe una variable puntual (p. ej. TF_VAR_system_prompt)

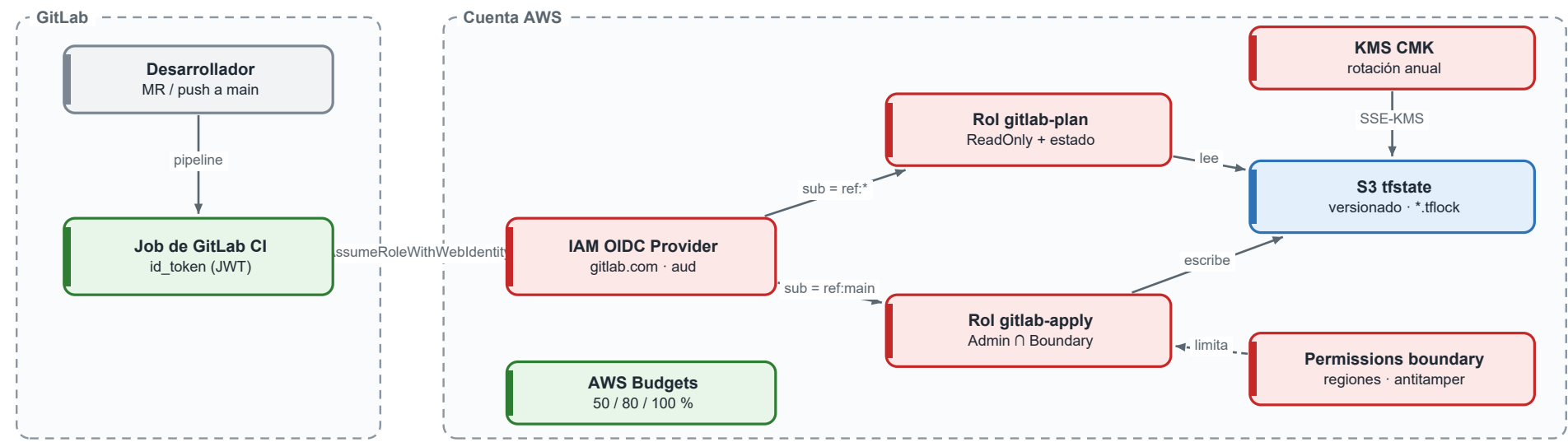
CI/CD configuration file: 03-cloud-e-infraestructura/lab10-terraform-aws/.gitlab-ci.yml · Protected branch: main (merge: Maintainers, push: nadie) · Protected environments: dev/*, prod/* · Pipeline must succeed + 1 aprobación + CODEOWNERS.

Parte 2 · Arquitecturas e implementación por micro lab

Cada lab muestra el diagrama (colores por categoría, fronteras punteadas = cuenta / VPC / subnet), la tabla de cómo se implementa con Terraform y los datos de operación.

Micro lab 00 · Bootstrap: estado remoto, GitLab OIDC y guardrails de costo

Prepara la cuenta para desplegar todo desde GitLab sin llaves estáticas: estado cifrado, roles separados para plan y apply, permissions boundary y presupuesto.



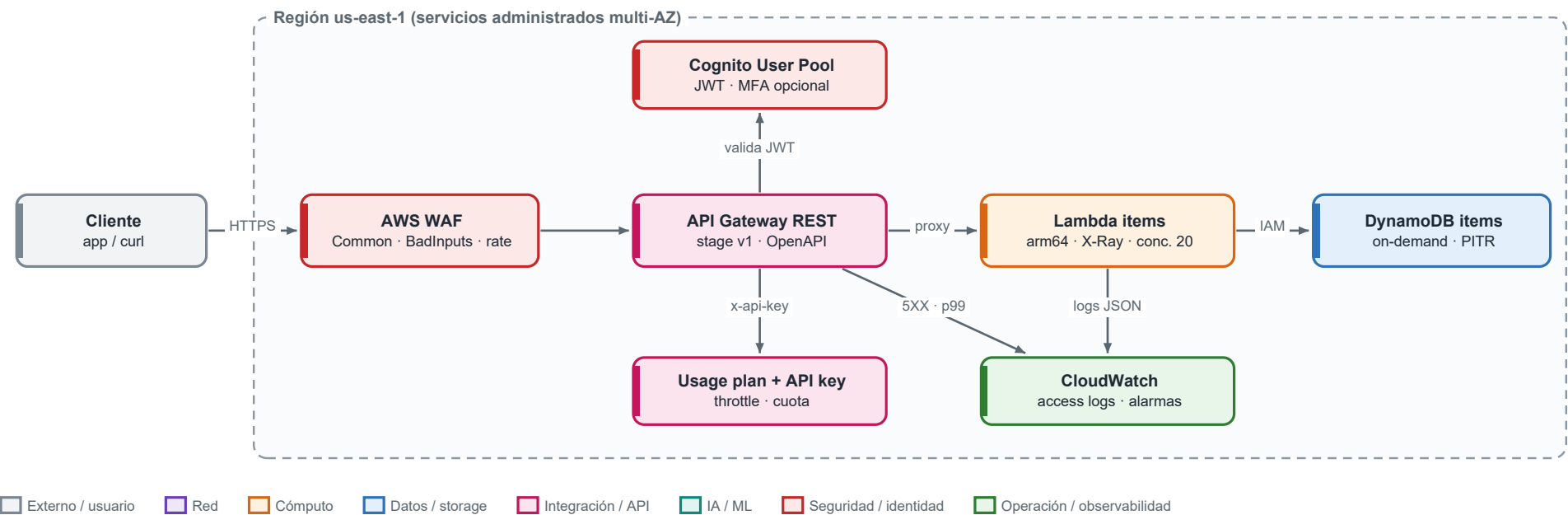
Externo / usuario Red Computo Datos / storage Integración / API IA / ML Seguridad / identidad Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Llave de cifrado	module.state_key → modules/kms-key	alias = lab10-dev-tfstate; enable_key_rotation = true
Bucket de estado	module.state_bucket → modules/s3-secure-bucket	force_destroy = false; versionado; noncurrent 90 días; política TLS-only; BPA
Proveedor OIDC	aws_iam_openid_connect_provider.gitlab	url = https://gitlab.com; client_id_list = [gitlab_audience]
Rol de plan	aws_iam_role.plan + ReadOnlyAccess + state_access	Condición sub: project_path:<grupo/proyecto>;ref_type:branch:ref:*
Rol de apply	aws_iam_role.apply + AdministratorAccess	sub = ...:ref:main; permissions_boundary = boundary; sesión de 1 h
Boundary	aws_iam_policy.boundary	Deny fuera de allowed_regions; deny CloudTrail/GuardDuty/Config stop; deny iam:CreateUser/AccessKey
Presupuesto	aws_budgets_budget.monthly	monthly_budget_usd = 50; alert_emails; 50/80 % real, 100 % pronosticado
Backend	versions.tf (local → s3)	terraform init -migrate-state con use_lockfile = true (sin DynamoDB)
Variables GitLab	TF_STATE_BUCKET, AWS_PLAN_ROLE_ARN, AWS_APPLY_ROLE_ARN (protegida), AWS_REGION	

Storage mínimo	S3 < 1 MB por lab; 1 CMK (~USD 1/mes)
Well-Architected	SEC: OIDC + boundary · OPS: IaC + estado versionado · COST: Budgets
Prueba automatizada	bash scripts/lab.sh test 00-platform-bootstrap-s3-oidc → bucket seguro, OIDC por rama, boundary niega iam.CreateUser

Micro lab 01 · API serverless: API Gateway REST + Lambda + DynamoDB

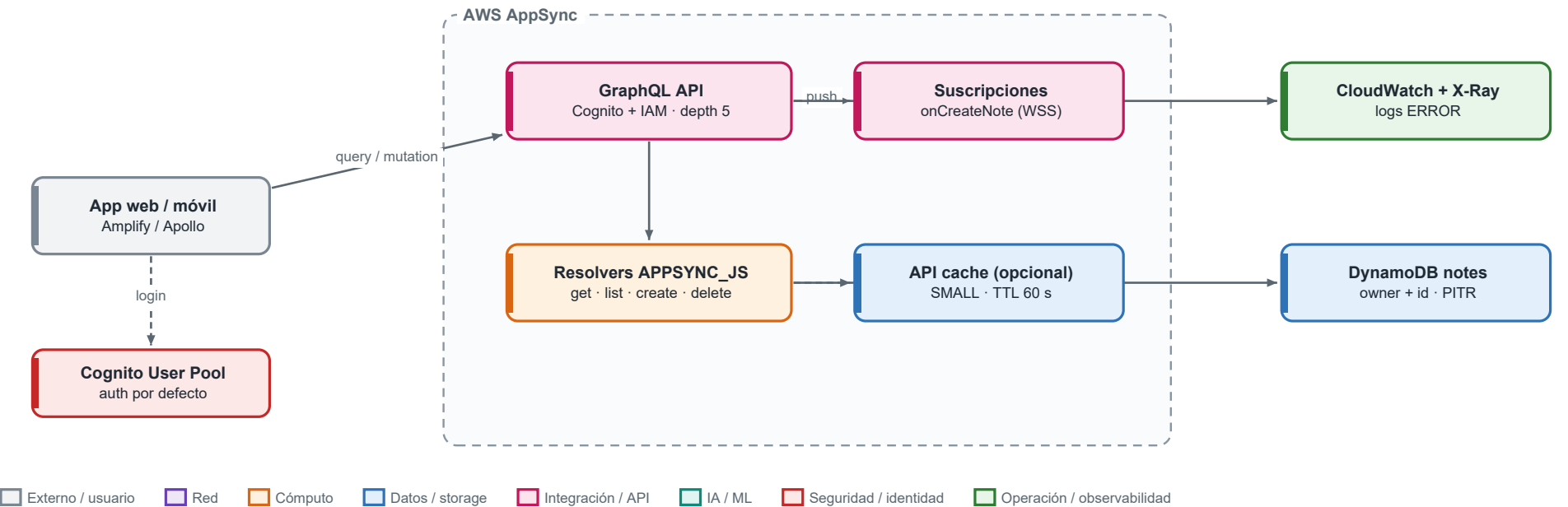
API REST definida por contrato OpenAPI, con defensa en profundidad (WAF, API key, JWT de Cognito y validación de esquema) y observabilidad completa.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Contrato API	openapi.yaml.tpl → templatefile()	Rutas /health (MOCK), /items, /items/{id}; request validator; securitySchemes cognito + api_key
API	aws_api_gateway_rest_api / _deployment / _stage	REGIONAL; redeploy por sha1(body); xray_tracing_enabled; access_log_settings JSON
Throttling	aws_api_gateway_method_settings + _usage_plan	throttle_rate_limit = 50, burst = 100, monthly_quota = 100000; data_trace off
Identidad	aws_cognito_user_pool / _client	Password de 12+ caracteres, MFA TOTP opcional, tokens de 60 min, prevent_user_existence_errors
Cómputo	module.items_fn → modules/lambda-function	python3.13 arm64; reserved_concurrency = 20; política inline solo sobre la tabla
Datos	module.table → modules/dynamodb-table	pk/sk (USER#sub / ITEM#uuid); PAY_PER_REQUEST; PITR; SSE
Borde	aws_wafv2_web_acl + _association	enable_waf; waf_rate_limit_per_5min = 1000; reglas administradas de AWS
Alarmas	aws_cloudwatch_metric_alarm (5xx, latency_p99)	5XX > 5 en 3 de 5 min; p99 > 1500 ms
Variables GitLab	LAB01_TFVARS (File)	
Storage mínimo	DynamoDB on-demand; logs de 30 días; paquete Lambda < 1 MB	
Well-Architected	SEC: 5 capas de control · REL: throttling + PITR · COST: pago por uso + cuotas	
Prueba automatizada	bash scripts/lab.sh test 01-serverless-apigw-lambda-dynamodb → 200/401/403/400/201/204/404, throttling, logs y X-Ray	

Micro lab 02 · AWS AppSync (GraphQL) + DynamoDB + suscripciones en tiempo real

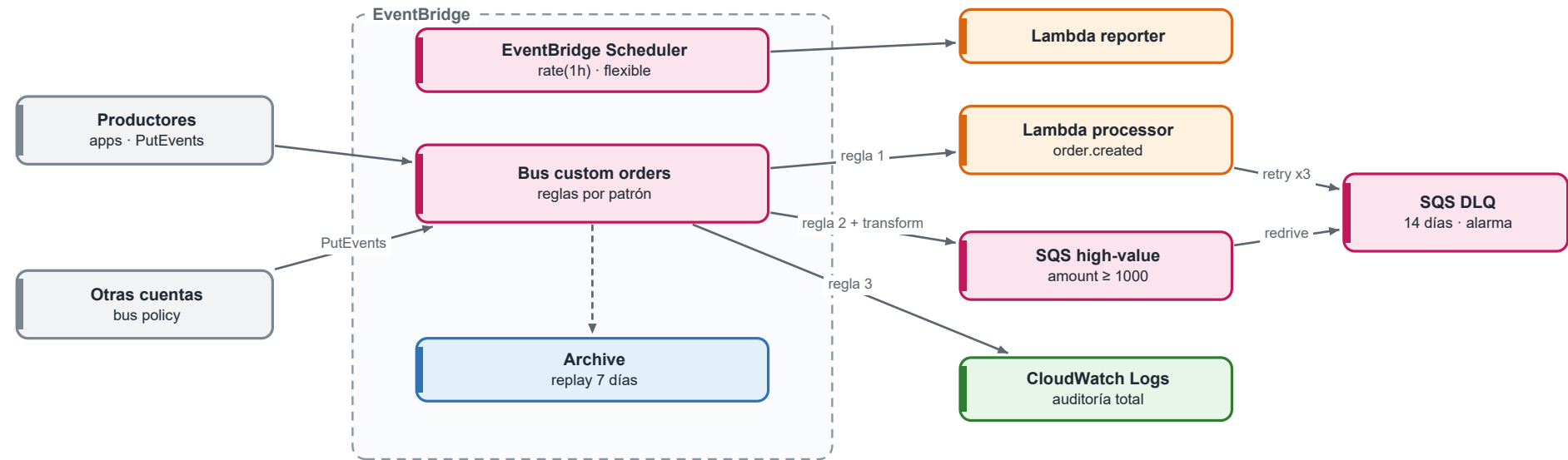
API GraphQL administrada con resolvers JavaScript que acceden directo a DynamoDB (sin Lambda), autorización por usuario y WebSockets.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Schema	schema.graphql → file()	Directivas @aws_cognito_user_pools / @aws_iam; @aws_subscribe(mutations: [createNote])
API	aws_appsync_graphql_api	AMAZON_COGNITO_USER_POOLS + AWS_IAM; query_depth_limit = 5; introspection DISABLED en prod; xray
Logs	log_config + aws_iam_role.logs	field_log_level = ERROR; exclude_verbose_content = true; retención de 30 días
Data source	aws_appsync_datasource.notes	AMAZON_DYNAMODB con rol limitado a Get/Put/Delete/Query de la tabla
Resolvers	aws_appsync_resolver (for_each local.resolvers)	runtime APPSYNC_JS 1.0.0; código en resolvers/*.js; filtran por ctx.identity.sub
Datos	module.notes_table	hash owner, range id; on-demand; PITR
Caché	aws_appsync_api_cache (count)	enable_cache = false; FULL_REQUEST_CACHING; cifrado en tránsito y en reposo
Variables GitLab	LAB02_TFVARS (File)	
Storage mínimo	DynamoDB on-demand + PITR; logs de 30 días	
Well-Architected	PERF: sin cold starts · SEC: autorización por campo · COST: sin Lambda intermedia	
Prueba automatizada	bash scripts/lab.sh test 02-serverless-appsync-graphql → aislamiento entre 2 usuarios, depth limit, 401	

Micro lab 03 · Arquitectura orientada a eventos con Amazon EventBridge

Desacoplar productores y consumidores: filtrado por contenido, transformación, reintentos, DLQ, archive/replay, bus cross-account y Scheduler.

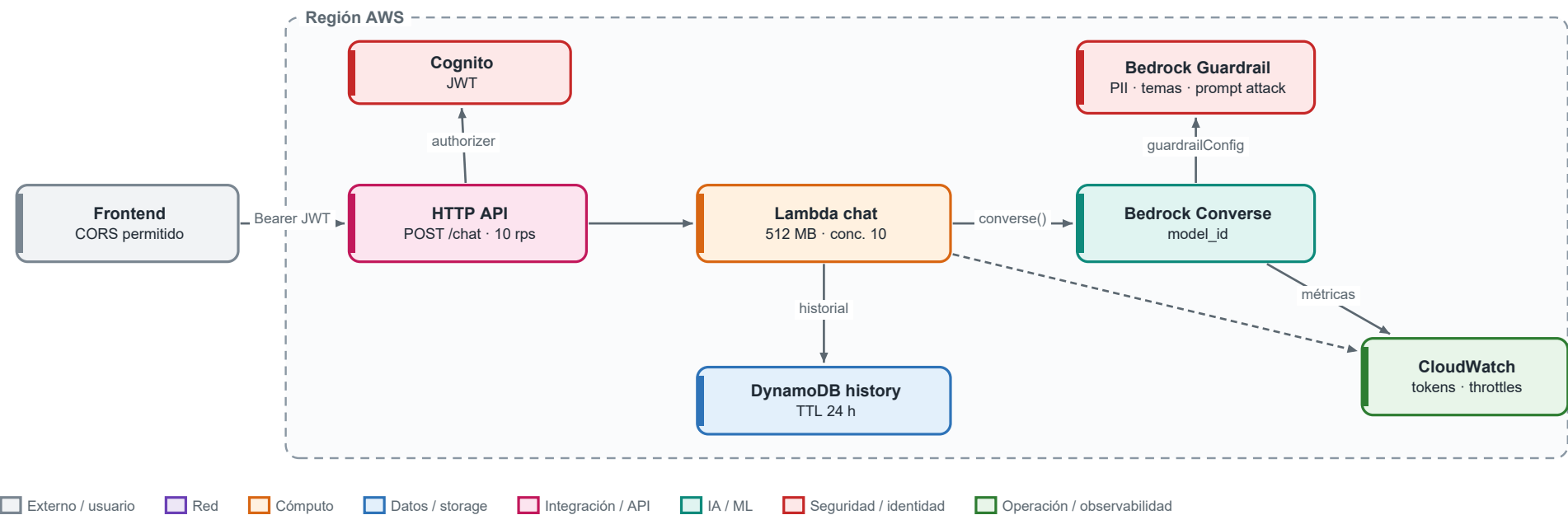


Externo / usuario Red Computo Datos / storage Integración / API IA / ML Seguridad / identidad Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Bus y archive	aws_cloudwatch_event_bus / _event_archive	archive_retention_days = 7; replay desde la consola o la CLI
Cross-account	aws_cloudwatch_event_bus_policy (count)	producer_account_ids = [] → events:PutEvents por cuenta
Regla 1	aws_cloudwatch_event_rule + _target	source com.lab10.orders, detail-type order.created; retry 3, edad máx. 1 h, DLQ
Regla 2	event_pattern numeric	detail.amount ≥ high_value_threshold; input_transformer → JSON reducido a SQS
Regla 3	target CloudWatch Logs + log resource policy	Log group /aws/events/...-audit; retención log_retention_days = 14
Colas	aws_sqs_queue (high_value, dlq) + policias	SSE-SQS; políticas con aws:SourceArn de la regla; maxReceiveCount = 5
Falla de código	aws_lambda_function_event_invoke_config	processor_async_retries = 1; on_failure → DLQ (la DLQ del target solo cubre fallas de entrega)
Scheduler	aws_scheduler_schedule	report_schedule = rate(1 hour); zona horaria America/Tijuana; ventana flexible de 15 min; DLQ
Alarmas	dlq_not_empty, failed_invocations	Mensajes en la DLQ > 0; FailedInvocations > 0
Variables GitLab	LAB03_TFVARS (File)	
Storage mínimo	Archive de 7 días; DLQ de 14 días; logs de 14 días	
Well-Architected	REL: retry + DLQ + replay · SEC: SourceArn · COST: filtrar en el bus	
Prueba automatizada	bash scripts/lab.sh test 03-events-eventbridge-sqs → ruteo, transformación, auditoría y DLQ por falla de código	

Micro lab 04 · Chatbot de IA generativa con Amazon Bedrock

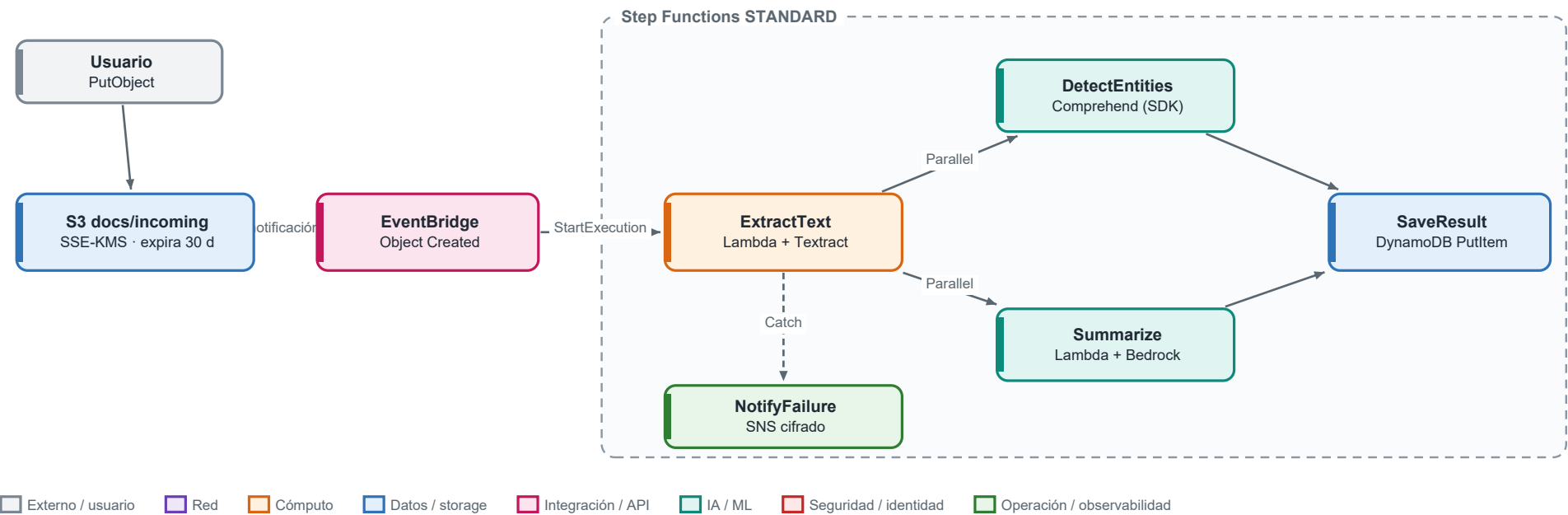
Chatbot serverless con memoria conversacional, autenticación JWT, Bedrock Guardrails y controles de costo sobre tokens y concurrencia.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Guardrail	aws_bedrock_guardrail + _version	Filtros HATE/INSULTS/SEXUAL/VIOLENCE/MISCONDUCT HIGH; PROMPT_ATTACK; PII anonymize/block; tema DENY
Modelo	variable model_id	amazon.nova-lite-v1:0 por defecto; Converse API (cambias de modelo sin tocar el código)
Lambda	module.chat_fn	timeout = 60; memory = 512; reserved_concurrency = max_concurrency (tope de gasto)
Política IAM	data.aws_iam_policy_document.chat	bedrock:InvokeModel (foundation-model, inference-profile); ApplyGuardrail; DynamoDB de la tabla
Prompt y límites	environment SYSTEM_PROMPT, MAX_TOKENS, HISTORY_TURNS	max_tokens = 512; history_turns = 6; system_prompt parametrizable desde GitLab
API	aws_apigatewayv2_api / _authorizer / _route / _stage	JWT con issuer de Cognito; CORS allowed_origins; throttling_rate_limit = 10
Historial	module.history	session_id = sub#session, ts (N); ttl_attribute = expires_at
Costo	alarms throttles + output_tokens	hourly_output_token_alarm = 200000 tokens/h
Variables GitLab	LAB04_TFVARS (File), TF_VAR_system_prompt	
Storage mínimo	DynamoDB con TTL 24 h; logs de 30 días	
Well-Architected	Generative AI Lens · SEC: guardrails + JWT · COST: tokens y concurrencia acotados	
Prueba automatizada	bash scripts/lab.sh test 04-genai-chatbot-bedrock → respuesta, memoria (4 ítems), guardrails tema + prompt injection	

Micro lab 05 · Pipeline de IA para documentos (Step Functions + Textract + Comprehend + Bedrock)

Orquestar servicios de IA administrados: integraciones SDK directas, paralelismo, reintentos con jitter y manejo de errores centralizado.



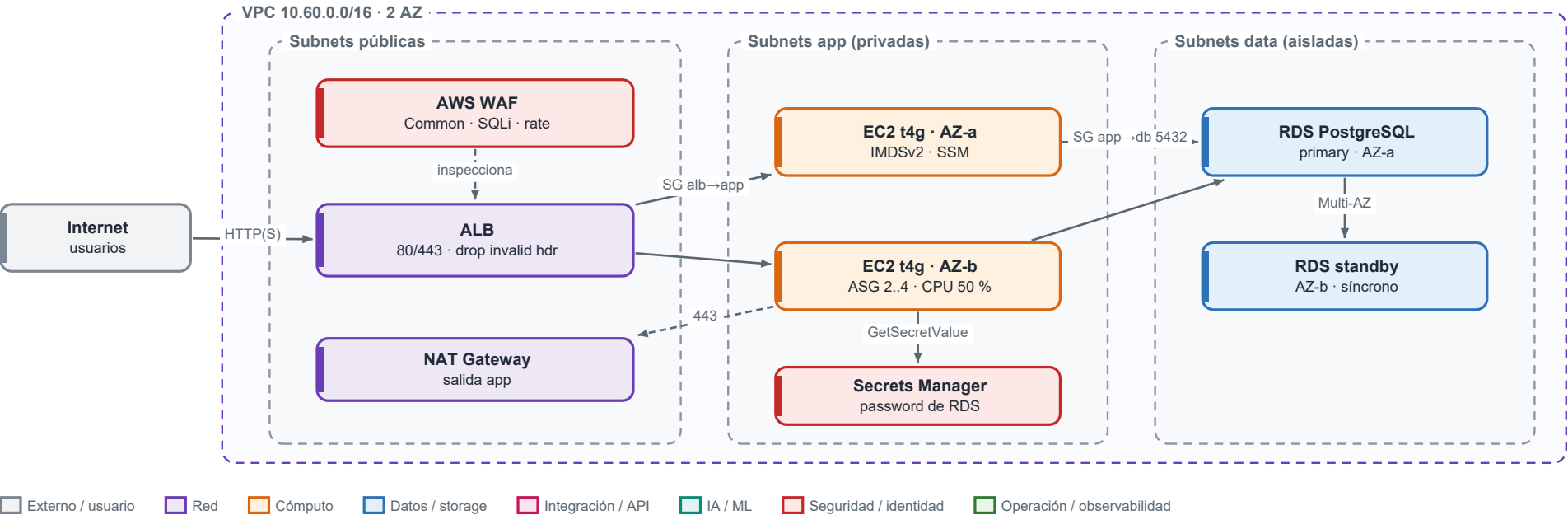
Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Flujo	statemachine.asl.json.tpl → templatefile()	ASL con \${extract_fn_arn}, \${summarize_fn_arn}, \${table_name}, \${alerts_topic_arn}, \${language_code}
Máquina de estados	aws_sfn_state_machine.pipeline	STANDARD; logging ERROR con include_execution_data = false; tracing activo
Disparador	aws_s3_bucket_notification (eventbridge=true) + event rule	Filtro bucket.name y object.key prefix incoming/; rol events → states:StartExecution
Extracción	module.extract_fn	Textract DetectDocumentText (png/jpg/pdf) o lectura de .txt; MAX_CHARS = 4500
Entidades	Integración arn:aws:states:::aws-sdk:comprehend:detectEntities	language_code = es; sin Lambda intermedia
Resumen	module.summarize_fn	Bedrock Converse; model_id; reserved_concurrency = 5; maxTokens 300
Errores	Retry (backoff + JITTER FULL) / Catch → SNS → Fail	alert_emails; alarma ExecutionsFailed > 0
Seguridad	module.kms + module.documents	CMK compartida S3/SNS; IAM s3:GetObject solo en incoming/*
Variables GitLab	LAB05_TFVARS (File)	
Storage mínimo	S3 expira a 30 días; 1 ítem DynamoDB por documento	
Well-Architected	REL: retry/catch · SEC: sin datos en logs · PERF: paralelismo	

Prueba automatizada

bash scripts/lab.sh test 05-genai-docs-stepfunctions → SUCCEEDED con entidades y resumen; ruta FAILED; prefijo ignorado

Micro lab 06 · Arquitectura web de 3 niveles (ALB + Auto Scaling + RDS Multi-AZ)

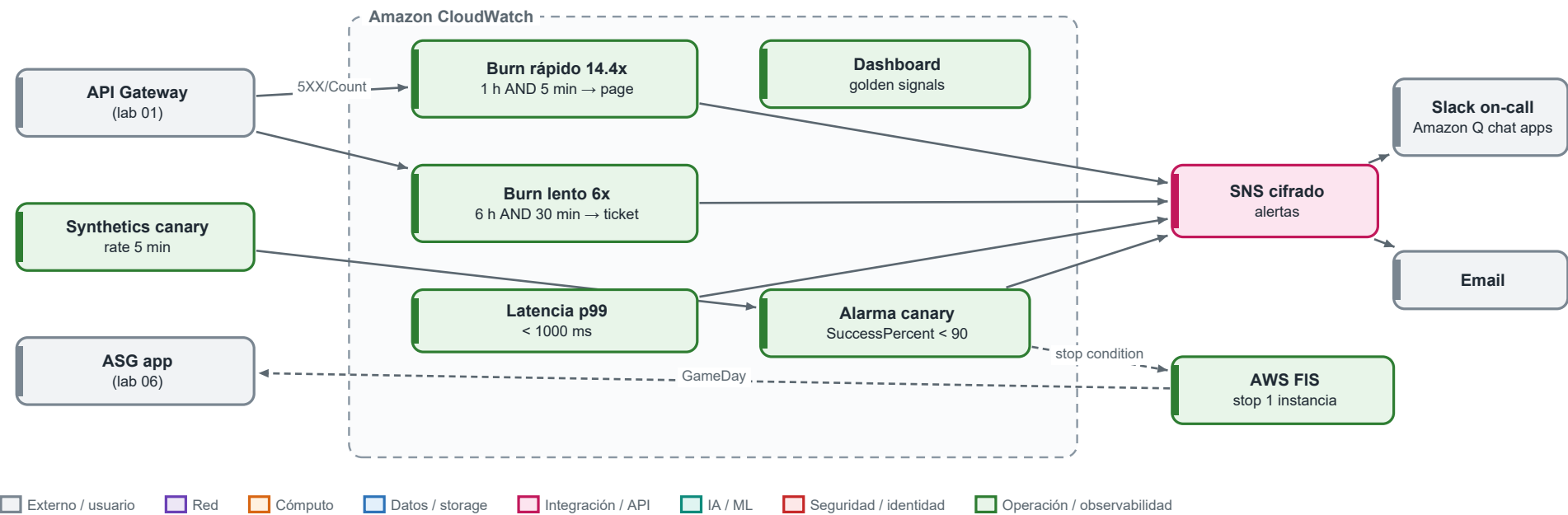
Arquitectura de referencia en VPC multi-AZ: un módulo por capa y seguridad encadenada (Internet → ALB → App → DB).



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Red	module.vpc → modules/vpc	vpc_cidr = 10.60.0.0/16; az_count = 2; single_nat_gateway = true (false en prod); flow logs REJECT
Capa web	./modules/web-tier	ALB + target group (/health); listener HTTP o HTTPS con certificate_arn; WAF Common + SQLi + rate 2000
Capa app	./modules/app-tier	Launch template AL2023 arm64 (SSM param); IMDSv2; EBS gp3 cifrado; ASG min/max; target tracking 50 %
App de prueba	user_data.sh.tpl (systemd lab10-app)	Python stdlib: /, /health, /db (lee el secreto, SQL por TLS, tabla visits); sin credenciales en user_data
Capa datos	./modules/data-tier	postgres 17; db.t4g.micro; 20 GB gp3 → 100 GB; multi_az; manage_master_user_password; rds.force_ssl
Cadena de SG	aws_vpc_security_group_ingress/egress_rule (root)	db_from_app y app_to_db en el root para evitar el ciclo entre módulos
Despliegue	instance_refresh Rolling	min_healthy_percentage = 50 → cambios sin caída
Alarmas	alb_5xx, unhealthy_hosts, db_cpu, db_storage	5XX > 10; hosts no sanos > 0; CPU > 80 %; FreeStorage < 2 GiB
Variables GitLab	LAB06_TFVARS (File); job con timeout 1 h	
Storage mínimo	EBS 8 GB gp3 por instancia; RDS 20 GB gp3 + backups de 7 días	
Well-Architected	REL: multi-AZ total · SEC: SG encadenados, sin SSH · COST: ~USD 95/mes, destruir al terminar	
Prueba automatizada	bash scripts/lab.sh test 06-web3tier-alb-asg-rds → 2 AZ sanas, /db TLS vía Secrets Manager, RDS privado, WAF SQLi; CHAOS=1	

Micro lab 07 · SRE: SLOs, alertas por burn rate, Synthetics, ChatOps y Chaos Engineering

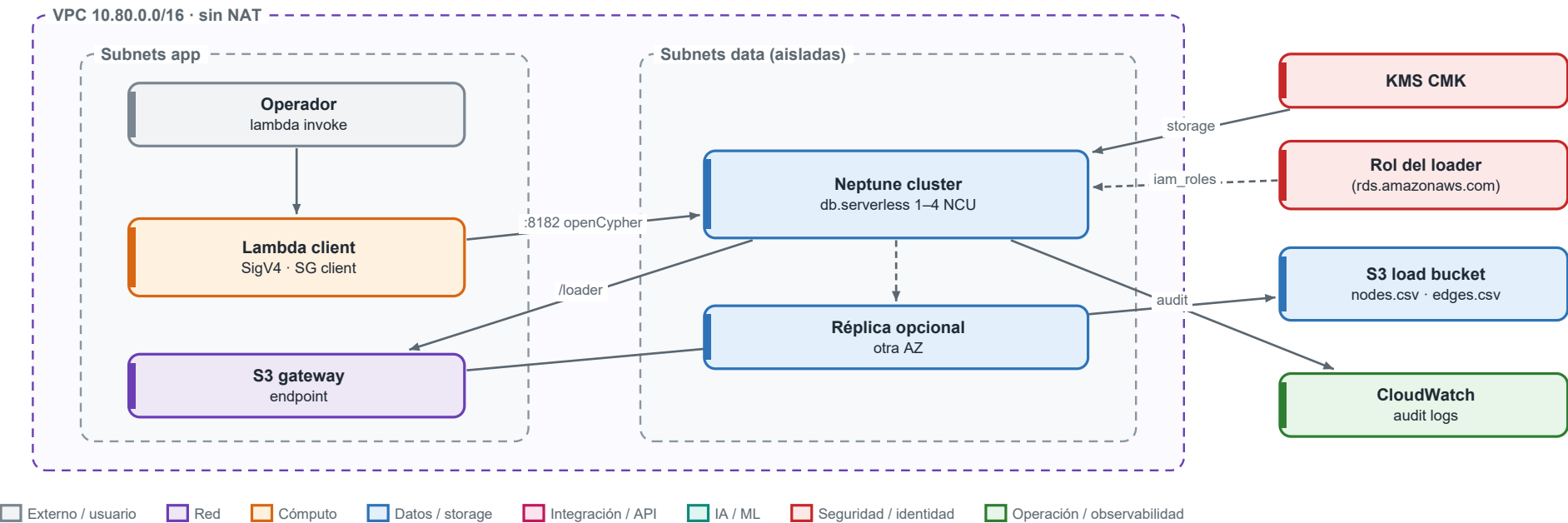
Alertar por consumo del presupuesto de error (multi-window, multi-burn-rate), monitorear desde fuera, notificar en Slack y validar resiliencia con AWS FIS.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
SLO	locals.error_budget, burn_windows	slo_target = 0.999 → 43 min/mes; fast 14.4x (1 h/5 min), slow 6x (6 h/30 min)
Burn rate	aws_cloudwatch_metric_alarm.burn_long / burn_short	metric_query: IF(requests>0, errors/requests, 0) > burn × presupuesto
Alerta	aws_cloudwatch_composite_alarm.slo_burn	ALARM(long) AND ALARM(short); descripción con runbook_url
Latencia	latency_slo	latency_p99_ms = 1000; 2 de 3 periodos de 5 min
Caja negra	aws_synthetic_canary.health + canary/.../index.js	health_check_url; canary_runtime_version; artefactos en S3 7/14 días
Dashboard	aws_cloudwatch_dashboard + dashboard.json.tpl	Tráfico, % 5XX, p50/p90/p99, saturación, éxito del canary
ChatOps	aws_chatbot_slack_channel_configuration (count)	slack_team_id / slack_channel_id; guardrail ReadOnlyAccess
Chaos	aws_fis_experiment_template (count)	enable_fis; aws:ec2:stop-instances COUNT(1) por tag MicroLab; reinicio en PT5M
Variables GitLab	LAB07_TFVARS (File); SLACK_* enmascaradas	
Storage mínimo	S3 de artefactos del canary con expiración de 14 días	
Well-Architected	OPS: SLO + runbooks + postmortems · REL: chaos engineering con stop condition	
Prueba automatizada	bash scripts/lab.sh test 07-sre-slo-cloudwatch-fis → canary PASSED, alarmas compuestas; GENERATE_ERRORS=1 dispara fast burn	

Micro lab 08 · Amazon Neptune (grafos) Serverless v2 con openCypher

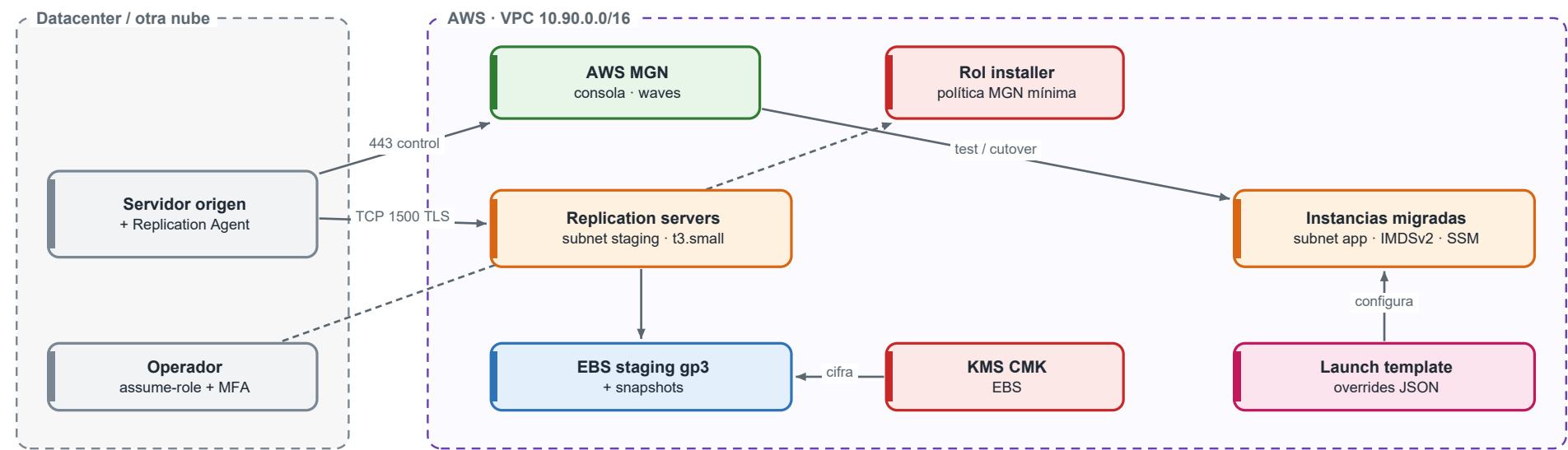
Base de grafos totalmente privada con autenticación IAM (SigV4), carga masiva desde S3 y consultas openCypher para fraude y recomendaciones.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Red	module.vpc	enable_nat_gateway = false; S3 gateway endpoint en las route tables de app y data
Cluster	aws_neptune_cluster	iam_database_authentication_enabled; storage_encrypted + KMS; backups de 7 días; audit logs
Escalado	serverless_v2_scaling_configuration	min_ncu = 1, max_ncu = 4; instance_class db.serverless; instance_count = 1 (2 = HA)
Parámetros	aws_neptune_cluster_parameter_group	neptune_family = neptune1.4 (debe coincidir con engine_version); audit log; timeout de query de 20 s
Seguridad de red	SG neptune ← SG client :8182	Sin CIDR abiertos: solo referencia entre security groups
Carga masiva	aws_iam_role.loader + aws_s3_object.sample	format opencypher; data/nodes.csv y edges.csv sincronizados por hash
Cliente	module.client_fn (vpc_subnet_ids)	neptune-db:connect / Read / Write / StartLoaderJob sobre cluster_resource_id
Variables GitLab	LAB08_TFVARS (File)	
Storage mínimo	Storage auto-escalable de 10 GB a 128 TiB (pago por GB-mes + I/O); backups de 7 días	
Well-Architected	SEC: sin Internet + IAM auth · COST: ~USD 115/mes con 1 NCU, destruir al terminar	
Prueba automatizada	bash scripts/lab.sh test 08-data-neptune-graph → bulk load, 8 nodos / 9 aristas, recomendación y fraude	

Micro lab 09 · Migración rehost con AWS Application Migration Service (MGN)

Landing zone de migración lift-and-shift: replicación continua, prueba, cutover y finalización, con credenciales temporales y cifrado KMS.

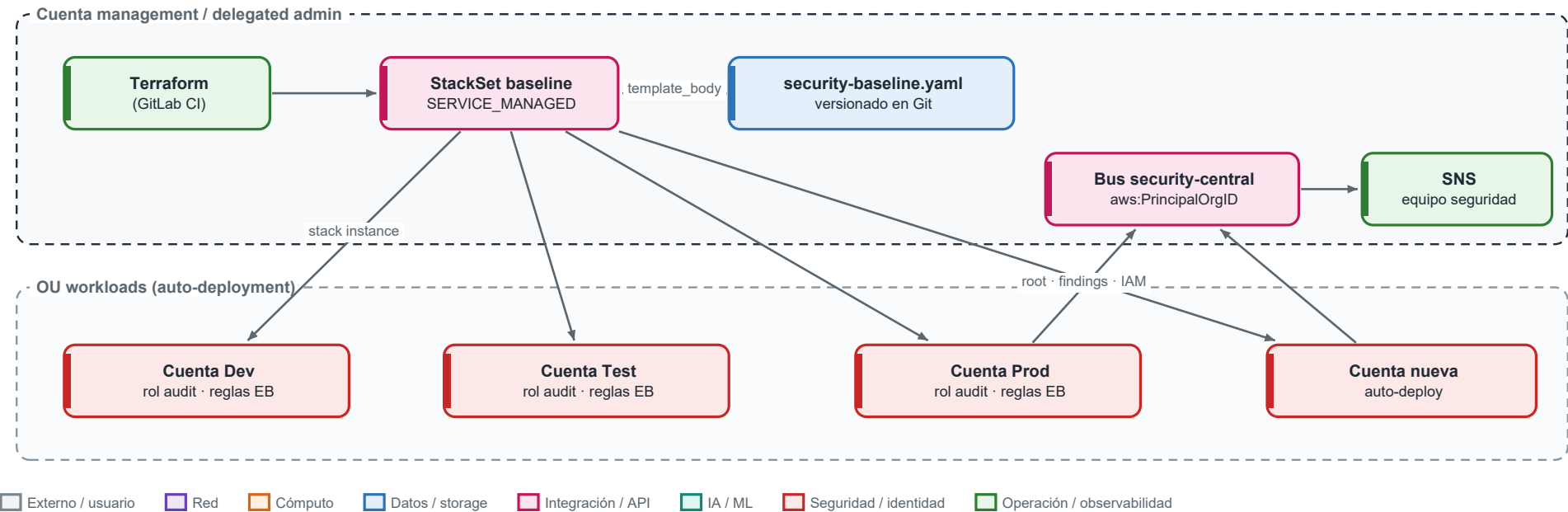


Externo / usuario Red Computo Datos / storage Integración / API IA / ML Seguridad / identidad Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Red	module.vpc + aws_subnet.staging	vpc_cidr sin traslape con on-prem; subnet de staging dedicada (cidrsubnet 30)
Replicación	aws_security_group.replication	Ingreso TCP 1500 solo desde source_cidrs; egreso 443
Ruta	replication_over_private_link	false = IP pública + TLS; true = VPN/Direct Connect (PRIVATE_IP)
Template de replicación	templates/replication-template.json.tpl → local_file	ebsEncryption CUSTOM + KMS; GP3; bandwidth_throttle_mbps; replication_server_instance_type
Launch template	templates/launch-template-overrides.json.tpl	Subnet y SG destino; iamInstanceProfile SSM; HttpTokens required; etiquetas
Aplicación de config	scripts/configure-mgn.sh (AWS CLI + jq)	initialize-service; update-replication-configuration-template; versiones de launch templates
Identidad	aws_iam_role.agent_installer	AWSApplicationMigrationAgentInstallationPolicy; MFA obligatoria; installer_principal_arns
Destino	aws_security_group.target + instance profile	target_app_ports = [80, 443] solo desde la VPC; SSM en lugar de SSH/RDP
Variables GitLab	LAB09_TFVARS (File); TF_VAR_source_cidrs	
Storage mínimo	EBS staging ≈ discos origen; EBS destino del mismo tamaño	
Well-Architected	Migration Lens · REL: RPO de segundos, rollback al origen · SEC: credenciales temporales	
Prueba automatizada	bash scripts/lab.sh test 09-migration-mgn → landing zone segura, estado de replicación MGN	

Micro lab 10 · CloudFormation StackSets: baseline multi-cuenta y multi-región

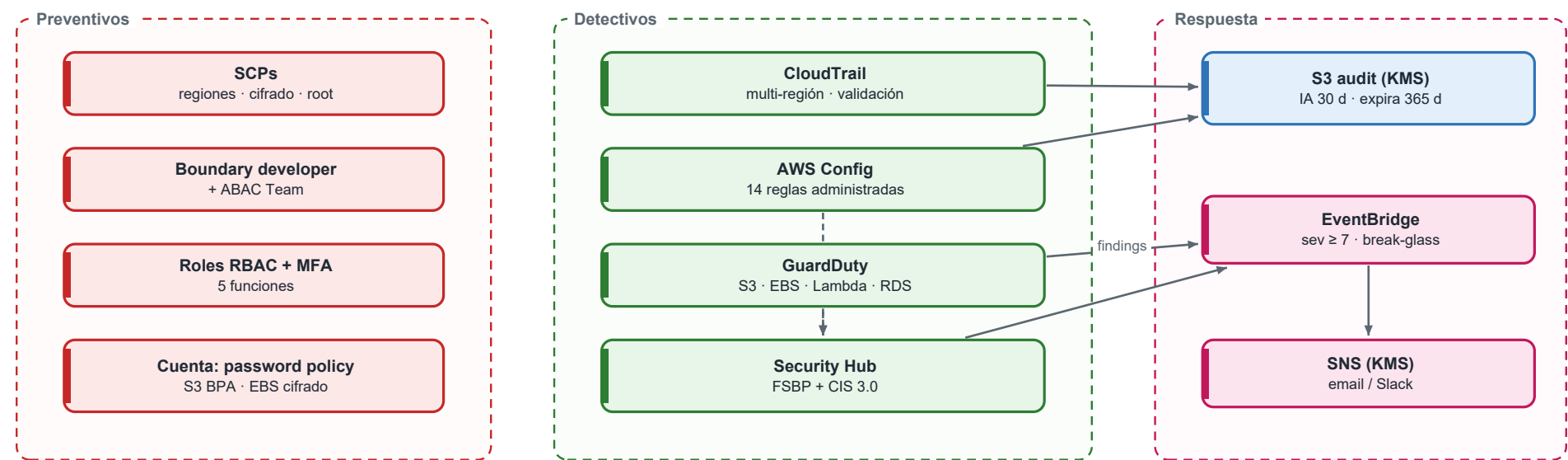
Gobernar N cuentas desde un punto: un StackSet SERVICE_MANAGED despliega el baseline de seguridad a las OUs y un bus central concentra los hallazgos.



Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
StackSet	aws_cloudformation_stack_set.baseline	permission_model SERVICE_MANAGED; call_as SELF / DELEGATED_ADMIN; CAPABILITY_NAMED_IAM
Despliegue	auto_deployment + operation_preferences	Cuentas nuevas reciben el baseline; max_concurrent_percentage = 25; failure_tolerance = 10 %
Instancias	aws_cloudformation_stack_set_instance.ous (for_each región)	deployment_targets.organizational_unit_ids = target_ou_ids; target_regions
Self-managed	stackset_admin + templates/stackset-execution-role.yaml	Alternativa sin Organizations: target_account_ids × target_regions
Baseline (CFN)	templates/security-baseline.yaml	Rol lab10-security-audit (MFA); reglas EB: root, GuardDuty ≥ 7, Security Hub HIGH, IAM; Config rules
Hub	aws_cloudwatch_event_bus.security + policy	organization_id restringe PutEvents; regla → SNS con input_transformer
Versionado	baseline_version	Subir la versión y hacer MR: el plan muestra el update y CloudFormation lo propaga gradualmente
Variables GitLab	LAB10_TFVARS (File); AWS_APPLY_ROLE_ARN_ORG (cuenta management)	
Storage mínimo	Ninguno (templates en Git)	
Well-Architected	Management & Governance Lens · OPS: drift detection · SEC: guardrails homogéneos	
Prueba automatizada	bash scripts/lab.sh test 10-governance-stacksets → StackSet ACTIVE, instancias SUCCEEDED, drift IN_SYNC	

Micro lab 11 · Seguridad y gobernanza: detectivos, preventivos, RBAC/ABAC y SCPs

Controles mínimos del pilar de Seguridad: registrar todo, detectar amenazas, medir cumplimiento (CIS/FSBP), prevenir configuraciones inseguras y controlar acceso por rol y atributo.



■ Externo / usuario ■ Red ■ Cómputo ■ Datos / storage ■ Integración / API ■ IA / ML ■ Seguridad / identidad ■ Operación / observabilidad

Componente	Recurso / módulo Terraform	Configuración clave (parámetros)
Auditoría	aws_cloudtrail.this + module.audit_bucket	is_multi_region_trail; enable_log_file_validation; KMS; audit_log_retention_days = 365
Config	recorder + delivery_channel + config_rule (for_each)	config_recording_frequency DAILY/CONTINUOUS; record_global_resources en una región; mapa config_managed_rules
Amenazas	aws_guardduty_detector + _detector_feature	guardduty_features = S3_DATA_EVENTS, EBS_MALWARE_PROTECTION, LAMBDA_NETWORK_LOGS, RDS_LOGIN_EVENTS
Cumplimiento	aws_securityhub_account + standards_subscription	securityhub_standards = FSBP 1.0.0, CIS 3.0.0; control_finding_generator SECURITY_CONTROL
Preventivos de cuenta	password_policy, s3_account_public_access_block, ebs_encryption_by_default	14+ caracteres, reuso 24, rotación 90 días; BPA total; EBS cifrado
RBAC	aws_iam_role.rbac (for_each var.rbac_roles)	break-glass 1 h, platform 4 h, developer 8 h + boundary, auditor, finops; MFA de menos de 1 h
ABAC	policies/abac-ec2-team.json	aws:ResourceTag/Team = aws:PrincipalTag/Team; etiqueta obligatoria al crear
SCPs	aws_organizations_policy (manage_scps)	policies/scp/*.json → scp_target_ids (OU sandbox primero)
Variables GitLab	LAB11_TFVARS (File); aplicar antes que los workloads	
Storage mínimo	S3: Standard → IA a 30 días → expira a 365; snapshots de Config diarios	
Well-Architected	SEC 1-6 completos: identidad, detección, infraestructura, datos e incidentes	

Prueba automatizada

bash scripts/lab.sh test 11-security-guardrails-rbac → detectivos, preventivos, simulación RBAC, hallazgo de muestra

Anexo · Matriz Well-Architected por micro lab

Resumen de cómo cada lab aterriza los seis pilares. El detalle está en la sección Well-Architected de cada README.

Lab	Excelencia operativa	Seguridad	Confiabilidad	Eficiencia de rendimiento	Costos	Sostenibilidad
00	IaC + estado versionado	OIDC, boundary, KMS	Versionado del estado	N/A	Budgets	Sin recursos ociosos
01	OpenAPI, logs JSON	WAF, Cognito, API key	Throttling, PITR	arm64, MOCK	Pago por uso	Graviton
02	Schema en Git	Auth por campo	Condiciones DDB	Sin Lambda	Sin cómputo extra	Menos saltos
03	Auditoría + replay	SourceArn	Retry + DLQ	Filtro en el bus	USD 1/M eventos	Por evento
04	Tokens en logs	Guardrails + JWT	Alarmas de throttling	Converse API	Topes de tokens	Modelo mínimo
05	Flujo visual	Sin datos en logs	Retry + Catch	Parallel + SDK	Concurrencia 5	Expiración
06	Módulos por capa	SG encadenados	Multi-AZ total	Target tracking	NAT único en dev	Graviton
07	SLO + runbooks	Chatbot RO	Chaos con FIS	p99 SLI	Canary de 5 min	Menos ruido
08	Audit logs	IAM auth, sin Internet	6 copias en 3 AZ	Serverless NCU	Sin NAT	Escala a demanda
09	Waves + test	MFA, KMS, temporales	RPO de segundos	Right-sizing	Finalizar rápido	Apagar on-prem
10	Drift, versión	Baseline homogéneo	Tolerancia a fallas	Paralelo	Sin costo	1 template para N cuentas
11	Hallazgos centralizados	SEC 1–6	Logs inmutables	N/A	Config DAILY	Ciclo de vida de logs

Orden recomendado: 00 → 11 → 01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10. Destruye los labs 06 (~USD 95/mes) y 08 (~USD 115/mes) al terminar cada sesión.